

Bookstore Class Diagram

Design Rationale & Future Considerations

CEN 4065 - Software Architecture & Design

Purpose

This document records design decisions made while developing the bookstore UML class diagram. It explains why selected classes, abstractions, and relationships were used and identifies areas where the design can expand as additional requirements are introduced. The goal is not to model every possible commercial bookstore feature, but to establish a modular foundation around users, books, carts, orders, payments, inventory, shipping, and returns.

1. Physical Books and EBooks

Book is modeled as a general class with PhysicalBook and EBook as specialized subclasses. Both share stable information such as ISBN, title, author, genre, and price, but their fulfillment behavior differs.

A PhysicalBook may require inventory tracking and shipment. An EBook does not require physical stock or shipping and instead becomes digitally available after payment. This behavioral difference justifies inheritance more strongly than a simple bookType attribute.

2. Inventory Remains Separate from Book

Inventory describes a physical book but has a different responsibility and rate of change from bibliographic information. A title or ISBN is relatively stable, while stock quantity may change on every sale or restock.

Keeping Inventory separate also creates room for future requirements such as multiple stores, warehouses, reorder thresholds, reserved stock, or low-stock notifications without redesigning Book.

3. ShoppingCart and CartItem

ShoppingCart represents the customer's temporary collection before purchase, while CartItem represents one book entry and its quantity. Quantity belongs to the relationship between a cart and a particular book, not to Book itself.

Composition is appropriate when CartItem has no useful independent lifecycle outside its ShoppingCart.

4. OrderItem Preserves the Purchase

OrderItem represents what was actually purchased, while CartItem represents what the customer currently intends to purchase. OrderItem can preserve transaction-specific data such as quantity and purchase price.

Preserving purchase price is important because a Book's current catalog price may change after an order is completed. OrderItem also provides a natural target for ReturnRequest because a return concerns a specific purchase rather than a generic catalog Book.

5. ShoppingCart and Order Stay Separate

ShoppingCart is temporary and editable; items can be added, removed, or cleared. Order represents a submitted transaction and must preserve historical information for order history, payment, fulfillment, and returns.

After checkout, a cart may be emptied and reused while the completed Order must continue to exist.

6. Payment vs. PaymentMethod

Payment represents the transaction itself: payment ID, amount, date, and status. PaymentMethod represents how the transaction is performed.

CreditCardPayment, DebitCardPayment, GiftCardPayment, and CashPayment can implement a common PaymentMethod interface. This avoids scattering payment-type conditionals throughout checkout and refund logic and provides a clean Strategy-style extension point.

7. PaymentMethod as an Extension Point

If future requirements add PayPal, Apple Pay, or another payment mechanism, a new implementation can be introduced without fundamentally redesigning Order or Payment.

The important goal is reasonable extensibility, not implementing unsupported payment types now.

8. SOLID Principles in the Diagram

Single Responsibility Principle (SRP): ShoppingCart, Order, Payment, Inventory, Shipment, ReturnRequest, and Book each focus on a separate domain responsibility rather than concentrating behavior in one large system class.

Open/Closed Principle (OCP): PaymentMethod allows new payment implementations to be added with minimal changes to existing payment logic.

Liskov Substitution Principle (LSP): PhysicalBook and EBook should be usable as Book objects wherever only common Book behavior is required. Subclass-specific behavior, such as shipping, should remain outside the generic Book contract.

Interface Segregation Principle (ISP): PaymentMethod should remain small and contain operations genuinely shared by its implementations.

Dependency Inversion Principle (DIP): Higher-level payment functionality can depend on PaymentMethod rather than directly depending on every concrete payment class.

9. Shipment Belongs to Order

Shipment should be associated with Order rather than directly with Book. Shipment information only becomes meaningful after a purchase requiring physical fulfillment has been placed.

A reasonable initial multiplicity is Order 1 -> 0..1 Shipment because an EBook-only order requires no shipment. If future requirements support split shipments or multiple warehouses, this can evolve to 0..* shipments per order.

10. ReturnRequest Represents a Process

A return has its own identity and lifecycle rather than being a simple `returned=true` attribute. It can store request date, reason, status, and review outcome.

The customer initiates the request and an administrator reviews it. These are stages of the same return lifecycle, so `ReturnRequest` remains one domain object.

11. ReturnRequest References OrderItem

A return should reference the purchased `OrderItem` rather than only `Book`. The same `Book` may be purchased by many customers, in different orders, at different prices and dates.

The path `ReturnRequest -> OrderItem -> Book` preserves the transaction context needed for return eligibility and refunds.

12. Avoid an Administrator 'God Class'

The first version may place several administrator actions on `Administrator` because the user stories describe those actions. As the design grows, `Administrator` should represent the actor or role rather than implementing every subsystem itself.

If catalog management, inventory management, returns, or backup/restore become complex, specialized services can be introduced later. They should be added because responsibilities have become substantial, not merely to increase the number of classes.

13. Security Considerations

The model should not imply that sensitive information is stored as unrestricted plaintext. User should prefer `passwordHash` over a raw password field.

Payment implementations should avoid exposing raw CVV or unrestricted card information in the domain model. Authentication, encryption, authorization, and tokenized payment data can be introduced when the architecture reaches that level of detail.

14. Multiplicities Are Business Rules

Multiplicity is not decorative UML notation; each value makes a claim about the bookstore. Examples include `Customer 1 -> 0..* Order`, `Order 1 -> 1..* OrderItem`, `CartItem 0..* -> 1 Book`, and `Order 1 -> 0..1 Shipment`.

These values should be revisited when requirements become more detailed rather than copied from convention.

15. Use Composition Selectively

`ShoppingCart -> CartItem` and `Order -> OrderItem` are strong composition candidates because the item objects exist specifically as parts of their parent objects.

`CartItem -> Book` should remain an association because deleting a cart must not delete the bookstore's `Book` record.

16. Do Not Turn Every Noun into a Class

Future requirements may mention Address, Genre, Author, Publisher, Price, Search, Download, Warehouse, Notification, Discount, Coupon, Tax, and Receipt. These do not automatically deserve separate classes.

A new class becomes more justified when the concept develops its own identity, state, behavior, lifecycle, or meaningful relationships. This keeps the model driven by requirements instead of diagram size.

Potential Future Extensions

Possible Extension	When It Becomes Justified
BookCatalog / CatalogService	Useful if searching and administrative catalog management become complex enough that Customer, Administrator, or Book would otherwise take on too much responsibility.
AuthenticationService	Useful if login, password reset, account verification, and session management become architecturally important.
Address	A String is sufficient initially. A class becomes useful if multiple addresses, billing addresses, validation, countries, states, or address selection are required.
Shipping Strategy	Useful if multiple shipping methods or carriers are introduced and must be selected interchangeably.
DownloadService	Useful if secure digital fulfillment grows to include authorization, expiring links, access limits, or file-delivery rules.
ReturnPolicy	Useful if return windows, digital restrictions, product condition, or exceptions become sufficiently complex.
Notification / Observer	Useful if order, shipment, return, or inventory state changes must notify multiple unrelated listeners.
Database Backup Service	Useful once infrastructure-level backup and restore behavior is modeled separately from the System Administrator actor.

Overall Design Direction

The bookstore class diagram should be treated as a foundation rather than a finished representation of every future feature. The design should preserve high cohesion, low coupling, abstraction where real behavioral variation exists, clear ownership, traceability to requirements or domain evidence, and extensibility without premature complexity.

The current model therefore aims to be detailed enough to represent important bookstore behavior, modular enough to evolve throughout the semester, and simple enough that every class and relationship has a clear reason to exist.